LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders

BehnamGhader, Adlakha, Mosbach, Bahdanau, Chapados & Reddy, COLM 2024

Raphael Leonardi, Baptiste Pras

Université Paris-Saclay · IR 2 · paper presentation

November 13, 2026

The problem: the best language models only read to the left

- The strongest LLMs (Llama, Mistral, Qwen) are **decoders**: causal attention, each token only sees the tokens before it
- For an embedding we want one vector for the **whole** text, so every token should see every token
- Text embedders are mostly **encoders** like BERT, trained on a lot of labelled pairs
- Decoder embedders in 2024 (E5-Mistral) needed big supervised data, sometimes made by GPT-4

Question: can we turn any decoder into a good text encoder, without labels and with little compute?

What the spring pipeline does, and what this paper changes

Spring pipeline

- BM25, or a small encoder off the shelf (MiniLM 22M, bge-small 33M)
- the encoder was trained with supervised contrastive learning on many query and document pairs
- one vector per text, cosine, then fusion and reranking

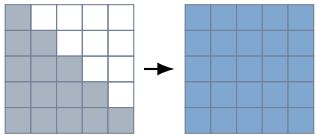
LLM2Vec

- starts from a decoder LLM, from 1.3B to 8B parameters
- three **unsupervised** steps, only raw text (Wikipedia)
- LoRA, 1000 steps per step, one GPU
- one vector per text (mean of the tokens), an instruction on the query

The search part does not change: it is still dense retrieval with cosine. Only the encoder changes.

The method: three steps

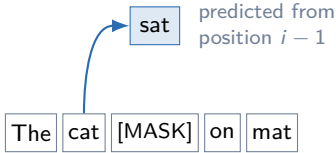
1. Bidirectional attention



causal all see all

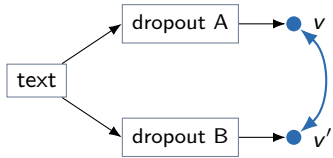
Only the attention mask changes, there is no training in this step.

2. MNTP



20% of the tokens are masked, like BERT. Each one is predicted from the output of the token **before** it, because the LLM head already predicts the next token.

3. Unsupervised SimCSE



The same text goes two times through the model with a different dropout: the two vectors are pulled together, the other texts of the batch (32) are pushed away.

Each step is a LoRA (rank 16), merged into the model before the next step. The text vector is the mean of the token vectors. The query gets an instruction, which is not in the mean.

Their evidence: the ablation (Table 5)

MTEB, 15 tasks	S-LLaMA 1.3B	Mistral 7B
Uni (the LLM as it is)	33.0	43.0
Bi	**30.3**	50.1
Bi + MNTP	42.1	53.9
Uni + SimCSE	45.0	52.0
Bi + SimCSE	44.5	60.3
Bi + MNTP + SimCSE	**52.4**	**60.5**

mean pooling, unsupervised, no labelled data

- **Claim:** the three steps together give the best unsupervised encoder
- Bi alone **hurts** most models, MNTP is there to repair it
- Mistral is the exception: Bi alone already helps
- **Speculated, not measured:** Mistral was maybe pretrained with some bidirectional attention

Our version, scaled down

- **Qwen2.5-0.5B** (494M) on a laptop: Mac M5, 24 GB, PyTorch MPS, bfloat16
- Same hyperparameters as the paper for S-LLaMA: LoRA $r = 16$, 1000 steps, batch 32
- Shorter texts: 128 tokens in training
- Training text: the **16,102 documents** of the three testbed corpora, never the queries or the qrels (the paper: Wikipedia)
- All six variants of Table 5, on SciFact and NFCorpus
- Then: Qwen2.5-1.5B (full recipe), and two encoders of 150M with the same SimCSE budget

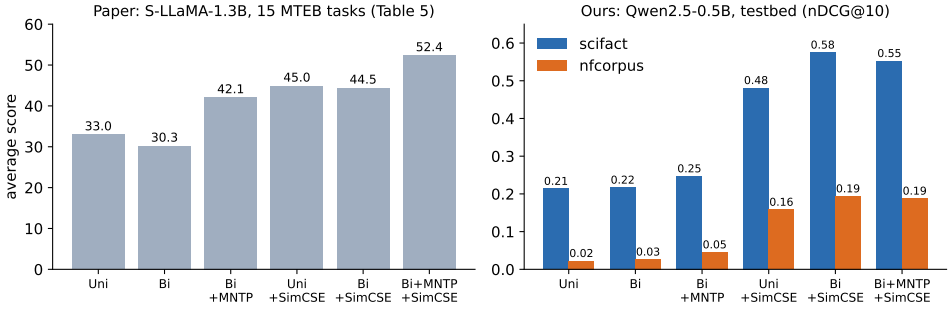
First a check of our code: the authors' own adapters, evaluated with our pipeline

Our number (1): the authors' adapters with our code

S-LLaMA-1.3B, MNTP + SimCSE adapters	SciFact	NFCorpus	ArguAna
Paper, Table 11 (MTEB)	38.3	15.7	43.6
Our code, testbed	37.9	16.0	44.3
Difference	−0.4	+0.3	+0.7

- Less than one point of difference, even if the testbed cuts documents after 200 words
- So our mask, instruction and pooling do the same as the authors' code
- When our own models are different from the paper, it comes from the **training**, not from the evaluation

Our number (2): the ablation on Qwen2.5-0.5B



- Same as the paper: **Bi alone does nothing**, SimCSE is the big step, Bi beats Uni after SimCSE ($+0.09$ SciFact, $p = 10^{-6}$)
- **Not the same:** MNTP before SimCSE gives nothing (-0.02, $p = 0.15$), in the paper it gives $+8$

Same budget for a real encoder? (the Ettin question)

nDCG@10	SciFact	NFCorpus	ArguAna
LLM2Vec Qwen 0.5B	**0.553**	**0.188**	**0.452**
Ettin-150m	0.089	0.021	
+ SimCSE, dropout 0.3	0.044	0.019	0.316
+ SimCSE, dropout 0.1	**0.003**	0.018	0.363
ModernBERT-base	0.063	0.019	
+ SimCSE, dropout 0.3	0.140	0.026	0.356
+ SimCSE, dropout 0.1	0.011	0.016	0.313

dropout 0.1 was tried after we saw the 0.3 results

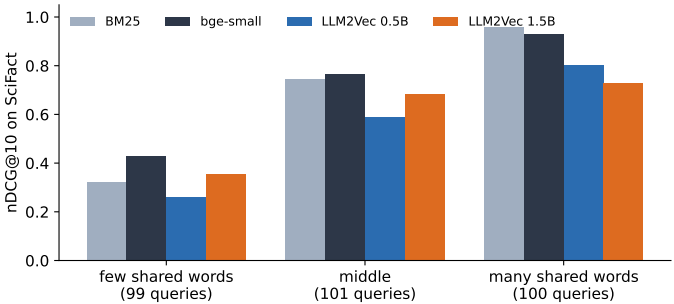
- Ettin paper: with the same data, encoders beat decoders made into encoders
- With **our** recipe, the encoders collapse on short queries
- The queries stay together (cosine 0.81 between queries) while documents spread (0.26)
- 300 SciFact queries get only **39 different documents** in their top 10. Qwen: 1939
- ArguAna survives: there the query is a whole argument, like a document

Our testbed row: LLM2Vec Qwen2.5-0.5B against bge-small

nDCG@10 (queries)	BM25	bge-small	**ours**	Δ vs bge	95% CI	p
SciFact (300)	0.676	0.710	**0.553**	-0.157	$[-0.197, -0.117]$	$< 10^{-12}$
NFCorpus (323)	0.321	0.340	**0.188**	-0.152	$[-0.178, -0.127]$	$< 10^{-25}$
ArguAna (1,406)	0.516	0.606	**0.452**	-0.154	$[-0.172, -0.134]$	$< 10^{-50}$

- We lose clearly against bge-small and BM25 on the three datasets
- Qwen 1.5B: 0.592, 0.218, 0.515. On ArguAna it is level with BM25 (-0.001, $p = 0.89$)
- bge-small has 33M parameters but was trained on a lot of **labelled** pairs; ours saw none
- The paper says it too: unsupervised LLM2Vec is far below its supervised version

Where we lose: claims that share many words with the answer



- Queries cut in three groups by the part of their words found in the labelled abstract
- Few shared words: we are close to BM25, the 1.5B model is **better than BM25** (+0.04)
- Many shared words: we lose **0.16** against BM25. Exact rare words (drugs, genes) get lost in one mean vector

What did not work, and what we did not test

Did not work

- MNTP before SimCSE: no gain, the main result we do not reproduce
- SimCSE on encoders with the same budget: they collapse, a smaller dropout is worse
- On MPS the default mask of transformers is not causal; and memory grows with each new batch length (we pad to 64)

Not tested

- Longer MNTP, or MNTP on Wikipedia like the paper
- Other pooling (last token, weighted mean)
- An encoder recipe for the encoders: SimCSE on short sentences
- Several seeds: one run per variant
- The ablation on the 1.5B model

14 paired tests on the ablation: with Bonferroni only $p < 0.0036$ counts, so 1.5B vs 0.5B ($p = 0.03$ and 0.005) is only a hint.

Compute budget

one Mac M5, 24 GB	hours
Qwen 0.5B: MNTP + 3 SimCSE	13.3
Qwen 1.5B: MNTP + SimCSE	14.4
2 encoders × 2 dropouts	8.4
Evaluation (35 runs of one dataset)	7.6
Total	**43.7**

- Not counted: tests, debugging, the authors' S-LLaMA training
- One full recipe on Qwen 0.5B: about 6 h
- Paper: 100 min of MNTP and 3 h of SimCSE for a 7B model on one A100

The two questions of the paper menu

Is causal attention the real bottleneck, or is it the pooling?

- Changing the mask alone does nothing (ours) or hurts (paper)
- It helps only **after** training: Bi + SimCSE beats Uni + SimCSE by 0.09 on SciFact
- Pooling: we only used the mean. In Table 5 the last token is always worse, by 4 to 27 points

A decoder trained on trillions of tokens loses to a 150M encoder, so why does the leaderboard say the opposite?

- In our test the decoder wins: the encoders did not survive the same recipe
- But a supervised 33M model (bge-small) beats all our models
- The leaderboard mostly measures the **training data and labels**, not only the architecture

Demo: claims on SciFact

rank of the labelled abstract (of 5,183)	BM25	bge-small	LLM2Vec
Hero: "0-dimensional biomaterials show inductive properties." No shared word	1745	13	**2**
Failure: "Insulin increases risk of severe kidney failure." All words shared	1	1	**77**

Documents encoded before the talk; only the claim is encoded live (0.06 s). Fallback recording: see demo.md.